



PROJECT REPORT

PROJECT II – IT3930

Facility location

Trần Tuấn Kiệt

Student ID: 20235132

Kiet.TT235132@sis.hust.edu.vn

Supervisors

Dr. Nguyễn Khánh Phương

Hanoi, 2026

Mục lục

1	Giới thiệu bài toán và thiết kế cấu trúc dữ liệu	4
1.1	Giới thiệu bài toán	4
1.2	Đối tượng Kho (Facility)	5
1.2.1	Các thuộc tính tĩnh	5
1.2.2	Các thuộc tính động	5
1.3	Đối tượng Khách hàng (Customer)	5
1.3.1	Các thuộc tính tĩnh	5
1.3.2	Các thuộc tính động	5
2	Mô hình toán học (MILP)	6
2.1	Tập chỉ số và Tham số	6
2.2	Biến quyết định	6
2.3	Hàm mục tiêu	6
2.4	Hệ thống các ràng buộc	7
2.4.1	Ràng buộc phục vụ khách hàng	7
2.4.2	Ràng buộc sức chứa của cơ sở	7
2.4.3	Ràng buộc chỉ gán cho cơ sở đã mở	7
2.4.4	Miền giá trị của biến	7
3	Phương pháp tiếp cận tham lam (Greedy algorithms)	8
3.1	Chiến lược Greedy 1: Ưu tiên mở cơ sở "đáng tiền nhất"	8
3.1.1	Ý tưởng thuật toán và tiêu chí tỷ lệ lợi ích/chi phí	8
3.1.2	Ưu điểm và nhược điểm của Greedy 1	9
3.2	Chiến lược Greedy 2: Ưu tiên gán khách hàng khó trước	9
3.2.1	Ý tưởng thuật toán và các tiêu chí đánh giá "mức khó"	10
3.2.2	Ưu điểm và nhược điểm của Greedy 2	10
4	Các phương pháp cải thiện nghiệm	12
4.1	Các bước di chuyển (Moves) cải thiện nghiệm	12
4.1.1	Move 1: Reassignment (Gán lại khách hàng)	12
4.1.2	Move 2: Close facility (Đóng cơ sở)	12
4.1.3	Move 3: Open + reassign (Mở và chuyển khách hàng)	13
4.1.4	Move 4: Swap (Hoán đổi trạng thái đóng/mở)	13
4.1.5	Move 5: Customer Swap (Hoán đổi hai khách hàng)	13
4.2	Sơ đồ thuật toán Local Search	14
4.3	Tối ưu hóa tính toán: Phương pháp tính nhanh Delta	14
4.4	Kết luận chương	15

5	CÀI ĐẶT THỰC NGHIỆM	16
5.1	Mô tả bộ dữ liệu chuẩn	16
5.2	Cài đặt mô hình Exact MILP sử dụng thư viện OR-Tools	17
5.3	Cài đặt các thuật toán Greedy	17
5.3.1	Greedy 1	17
5.3.2	Greedy 2	17
5.4	Cài đặt thuật toán Local Search	17
5.4.1	Thuật toán VND	18
5.4.2	Chạy thực nghiệm tổng hợp	18
5.5	Kết luận chương	18
6	SO SÁNH VÀ ĐÁNH GIÁ KẾT QUẢ THỰC NGHIỆM	19
6.1	So sánh chất lượng nghiệm giữa Exact MILP, Greedy và Local Search	19
6.2	So sánh thời gian thực thi của các phương pháp	21
6.3	Thảo luận về hiệu quả của các bước Local Search và phương pháp tính nhanh	23
6.3.1	Hiệu quả cải thiện so với Greedy 2	23
6.3.2	Hiệu quả của VND	23
6.3.3	Hiệu quả của phương pháp tính nhanh Delta	24
6.4	Kết luận chương	24
7	KẾT LUẬN	25
7.1	Tóm tắt các kết quả đạt được của dự án	25
7.1.1	Xây dựng được mô hình bài toán CFLP	25
7.1.2	Cài đặt mô hình Exact MILP bằng OR-Tools	25
7.1.3	Cài đặt các thuật toán Greedy	26
7.1.4	Cài đặt các bước Local Search	26
7.1.5	Cài đặt các phiên bản VND	26
7.1.6	Áp dụng phương pháp tính nhanh Delta	27
7.1.7	Kết quả tổng hợp	27
7.2	Định hướng phát triển và tối ưu hóa thêm	27
7.2.1	Cải thiện chiến lược khởi tạo nghiệm	28
7.2.2	Bổ sung các metaheuristic nâng cao	28
7.2.3	Tối ưu hóa tốc độ tính toán	28
7.2.4	Mở rộng bộ dữ liệu và so sánh chuyên sâu	28
7.3	Kết luận chung	28

Chương 1

Giới thiệu bài toán và thiết kế cấu trúc dữ liệu

1.1 Giới thiệu bài toán

Bài toán vị trí đặt kho có ràng buộc sức chứa (Capacitated Facility Location Problem - CFLP) bắt nguồn từ nhu cầu thực tế trong việc thiết kế mạng lưới chuỗi cung ứng. Ngữ cảnh điển hình là một công ty phân phối hàng tiêu dùng đang muốn xây dựng mạng lưới kho trung chuyển để cung cấp hàng cho các cửa hàng bán lẻ trong một thành phố.

Công ty đã khảo sát trước một tập hợp các vị trí ứng viên có thể mở kho. Mỗi vị trí ứng viên đặc trưng bởi một chi phí mở kho cố định và một sức chứa tối đa về lượng hàng có thể phục vụ trong một kỳ. Phân bố khắp thành phố là các cửa hàng bán lẻ (đóng vai trò là khách hàng), trong đó mỗi cửa hàng có một mức nhu cầu hàng hóa nhất định. Đồng thời, tùy thuộc vào khoảng cách và điều kiện giao thông, sẽ phát sinh một mức chi phí phục vụ tương ứng nếu cửa hàng đó được cung cấp hàng từ một kho ứng viên cụ thể.

Hệ thống cần đưa ra quyết định cốt lõi: nên mở những kho nào và mỗi cửa hàng sẽ được phục vụ bởi kho nào. Quá trình phân bổ này phải tuân thủ nghiêm ngặt hai ràng buộc:

- Mỗi cửa hàng được phục vụ bởi đúng một kho đã mở.
- Tổng nhu cầu của các cửa hàng được giao cho một kho không được phép vượt quá sức chứa tối đa của kho đó.

Mục tiêu tối thượng của mô hình là đảm bảo tổng chi phí mở kho và chi phí phục vụ đạt mức nhỏ nhất.

Để có thể cài đặt và thực thi hiệu quả các thuật toán tối ưu hóa (như thuật toán Tham lam hay Tìm kiếm cục bộ) cho bài toán này, việc tổ chức cấu trúc dữ liệu linh hoạt là yêu cầu tiên quyết. Dưới đây là thiết kế chi tiết cho hai đối tượng cốt lõi: Kho (Facility) và Khách hàng (Customer).

1.2 Đối tượng Kho (Facility)

Đối tượng này đại diện cho một vị trí ứng viên có thể mở kho trung chuyển. Việc quản lý tốt trạng thái của Kho sẽ giúp thuật toán kiểm tra nhanh chóng các ràng buộc về sức chứa.

1.2.1 Các thuộc tính tĩnh

Đây là các giá trị được khởi tạo một lần khi đọc từ file dữ liệu và không thay đổi trong suốt quá trình chạy thuật toán:

- **ID (Chỉ số):** Định danh duy nhất của kho ($i \in \{0, 1, \dots, I - 1\}$).
- **Capacity (Sức chứa tối đa - s_i):** Lượng hàng hóa tối đa mà kho có thể phục vụ trong một kỳ.
- **Fixed Cost (Chi phí mở kho - f_i):** Chi phí cố định phải bỏ ra nếu quyết định mở kho này.

1.2.2 Các thuộc tính động

- **Is Open (Trạng thái mở):** Biến logic xác định kho hiện tại đang được mở hay đóng tương đương với biến quyết định $y_i \in \{0, 1\}$.
- **Current Load (Tải trọng hiện tại):** Tổng nhu cầu của tất cả các khách hàng đang được gán cho kho này. Thuộc tính này dùng để kiểm tra điều kiện (Current Load + Nhu cầu khách mới $\leq$ Capacity, $\sum d_j * x_{ij} \leq s_i$) trước khi gán thêm khách hàng.
- **Residual Capacity (Sức chứa còn lại):** Bằng Capacity trừ đi Current Load. (Có thể tính toán trực tiếp hoặc lưu trữ riêng để truy xuất nhanh, $s_i - \sum d_j * x_{ij}$).
- **Assigned Customers (Danh sách khách hàng đang phục vụ):** Tập hợp các ID của khách hàng đang được gán cho kho này.

1.3 Đối tượng Khách hàng (Customer)

Đối tượng này đại diện cho một cửa hàng bán lẻ cần được cung cấp hàng hóa.

1.3.1 Các thuộc tính tĩnh

- **ID (Chỉ số):** Định danh duy nhất của khách hàng (ví dụ: $j \in \{0, 1, \dots, J - 1\}$).
- **Demand (Nhu cầu - d_j):** Lượng hàng hóa mà khách hàng này yêu cầu.
- **Serving Costs (Chi phí phục vụ - c_{ij}):** Một danh sách hoặc bản đồ (Map) lưu trữ chi phí vận chuyển từ TẤT CẢ các kho i đến khách hàng j này.

1.3.2 Các thuộc tính động

- **Assigned Facility (Kho đang phục vụ):** ID của kho mà khách hàng này hiện đang được gán tới (tương đương với việc tìm ra biến $x_{ij} = 1$).

Chương 2

Mô hình toán học (MILP)

Để giải quyết bài toán một cách chính xác, chúng ta có thể mô hình hóa bài toán Vị trí đặt kho có giới hạn sức chứa (Capacitated Facility Location Problem) dưới dạng Quy hoạch tuyến tính nguyên hỗn hợp (Mixed-Integer Linear Programming - MILP).

2.1 Tập chỉ số và Tham số

Mô hình sử dụng các tập chỉ số và tham số đầu vào như sau:

- I : tập cơ sở (kho) ứng viên, với chỉ số $i \in I$.
- J : tập khách hàng (cửa hàng bán lẻ), với chỉ số $j \in J$.
- f_i : chi phí cố định để mở cơ sở i .
- s_i : sức chứa tối đa của cơ sở i trong một kỳ].
- d_j : lượng hàng hóa nhu cầu của khách hàng j .
- c_{ij} : chi phí phục vụ (vận chuyển) để cung cấp hàng cho khách hàng j từ cơ sở i .

2.2 Biến quyết định

Bài toán yêu cầu xác định việc mở kho nào và phân bổ khách hàng ra sao, do đó sử dụng hai cụm biến quyết định nhị phân:

- $y_i \in \{0, 1\}$: bằng 1 nếu quyết định mở cơ sở i , ngược lại bằng 0.
- $x_{ij} \in \{0, 1\}$: bằng 1 nếu khách hàng j được gán (phục vụ) bởi cơ sở i , ngược lại bằng 0.

2.3 Hàm mục tiêu

Mục tiêu cốt lõi của bài toán là tối thiểu hóa tổng chi phí, bao gồm tổng chi phí mở cơ sở và tổng chi phí phục vụ khách hàng.

$$\min \sum_{i \in I} f_i y_i + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \quad (2.1)$$

2.4 Hệ thống các ràng buộc

Để đảm bảo tính hợp lệ của hệ thống thực tế, mô hình toán học phải thỏa mãn các ràng buộc sau đây:

2.4.1 Ràng buộc phục vụ khách hàng

Mỗi khách hàng phải được gán cho đúng một cơ sở duy nhất.

$$\sum_{i \in I} x_{ij} = 1 \quad \forall j \in J \quad (2.2)$$

2.4.2 Ràng buộc sức chứa của cơ sở

Tổng nhu cầu của tất cả các khách hàng được gán cho một cơ sở không được vượt quá sức chứa tối đa của cơ sở đó. Đồng thời, ràng buộc này cũng đảm bảo nếu cơ sở không mở ($y_i = 0$), sức chứa sẽ bằng 0 và không khách hàng nào được gán vào.

$$\sum_{j \in J} d_j x_{ij} \leq s_i y_i \quad \forall i \in I \quad (2.3)$$

2.4.3 Ràng buộc chỉ gán cho cơ sở đã mở

Một khách hàng chỉ có thể được gán cho một cơ sở nếu cơ sở đó đã được mở. Mặc dù ràng buộc này không bắt buộc tuyệt đối nếu đã có ràng buộc sức chứa ở trên (với $d_j > 0$), nhưng việc giữ lại ràng buộc này giúp mô hình chặt chẽ rõ ràng hơn và các bộ giải (solver) thường xử lý tốt hơn.

$$x_{ij} \leq y_i \quad \forall i \in I, \forall j \in J \quad (2.4)$$

2.4.4 Miền giá trị của biến

Các biến quyết định đều là biến nguyên nhị phân.

$$x_{ij} \in \{0, 1\}, \quad y_i \in \{0, 1\} \quad \forall i \in I, \forall j \in J \quad (2.5)$$

Chương 3

Phương pháp tiếp cận tham lam (Greedy algorithms)

Trong bài toán định vị cơ sở có ràng buộc sức chứa (Capacitated Facility Location Problem - CFLP), mục tiêu là lựa chọn một tập cơ sở để mở và phân bổ khách hàng cho các cơ sở đó sao cho tổng chi phí là nhỏ nhất. Tổng chi phí này bao gồm chi phí cố định khi mở cơ sở và chi phí phục vụ khách hàng từ cơ sở được chọn.

3.1 Chiến lược Greedy 1: Ưu tiên mở cơ sở "đáng tiền nhất"

Chiến lược Greedy 1 được xây dựng theo tư tưởng tham lam, tại mỗi bước thuật toán sẽ chọn cơ sở mang lại lợi ích tốt nhất ở thời điểm hiện tại. Cụ thể, thuật toán ưu tiên mở cơ sở có tỷ lệ lợi ích trên chi phí cao nhất, tức là cơ sở "đáng tiền nhất" để mở trong bước đó. Thay vì xét toàn bộ không gian nghiệm như mô hình tối ưu chính xác, phương pháp này ra quyết định từng bước dựa trên thông tin cục bộ. Sau khi mở một cơ sở, nhóm khách hàng chưa được phục vụ sẽ được gán cho cơ sở đó nếu còn đủ sức chứa, và quá trình lặp lại cho đến khi toàn bộ khách hàng được phân bổ.

3.1.1 Ý tưởng thuật toán và tiêu chí tỷ lệ lợi ích/chi phí

Ý tưởng chính của Greedy 1 là đánh giá từng cơ sở chưa mở dựa trên khả năng phục vụ các khách hàng còn lại.

Quy trình tổng quát được thực hiện như sau:

- Khởi tạo tất cả khách hàng ở trạng thái chưa được phân bổ và tất cả cơ sở ở trạng thái chưa mở.
- Với mỗi cơ sở chưa mở, xét các khách hàng chưa được phục vụ và lọc ra các khách hàng có nhu cầu không vượt quá sức chứa của cơ sở.
- Sắp xếp các khách hàng theo chi phí phục vụ tăng dần và chọn lần lượt cho đến khi hết sức chứa.
- Tính tỷ lệ lợi ích/chi phí của cơ sở đó và chọn cơ sở có tỷ lệ tốt nhất để mở.
- Gán các khách hàng đã chọn cho cơ sở vừa mở, lặp lại quá trình cho đến khi tất cả khách hàng được phân bổ.

Thuật toán ước lượng lợi ích của một cơ sở thông qua phần chi phí phục vụ tiết kiệm được. Mức tiết kiệm này là chênh lệch giữa chi phí phục vụ từ cơ sở đang xét với chi phí phục vụ lớn nhất của khách hàng đó trong toàn bộ các cơ sở. Công thức tính lợi ích và chi phí:

- Lợi ích: $saved_cost(i) = \sum (\max_k(c_{kj}) - c_{ij})$ với mọi khách hàng j thuộc tập được chọn tạm thời S_i .
- Chi phí: $cost(i) = f_i + \sum c_{ij}$ với f_i là chi phí cố định và c_{ij} là chi phí phục vụ.
- Tiêu chí lựa chọn (Tỷ lệ): $ratio(i) = \frac{saved_cost(i)}{cost(i)}$.

Mỗi quyết định mở cơ sở và gán khách hàng được thực hiện một lần và không quay lại chỉnh sửa, do đó nghiệm thu được phụ thuộc mạnh vào các lựa chọn ban đầu.

3.1.2 Ưu điểm và nhược điểm của Greedy 1

Ưu điểm:

- Thuật toán có cách triển khai đơn giản, dễ hiểu, dễ cài đặt để làm nền tảng cho các phương pháp cải tiến.
- Thời gian chạy nhanh hơn rất nhiều so với mô hình tối ưu chính xác (MILP), hữu ích khi kích thước bài toán lớn.
- Tiêu chí đánh giá giúp cân bằng giữa chi phí mở cơ sở và chi phí phục vụ khách hàng, tránh việc mở quá nhiều cơ sở đắt đỏ.
- Khả năng tạo ra nghiệm ban đầu khả thi để dùng làm đầu vào cho các thuật toán tìm kiếm cục bộ (Move 1, 2, 3, 4 hoặc VND).

Nhược điểm:

- Tính cục bộ trong quyết định rất cao, không xét đến ảnh hưởng của lựa chọn hiện tại tới các bước sau.
- Không có cơ chế sửa sai khi đã lỡ gán khách hàng, để làm nghiệm cuối cùng bị mắc kẹt.
- Tiêu chí tỷ lệ tính toán chỉ là thước đo xấp xỉ, không đảm bảo sẽ dẫn đến nghiệm toàn cục tốt nhất.
- Việc sắp xếp theo chi phí phục vụ tăng dần có thể bỏ qua yếu tố nhu cầu và sức chứa, gây khó khăn cho việc phân bổ dài hạn.
- Chất lượng nghiệm thường kém hơn so với Greedy 2 và các phương pháp cải thiện khác.

3.2 Chiến lược Greedy 2: Ưu tiên gán khách hàng khó trước

Một khó khăn trong bài toán CFLP là nếu các khách hàng có nhu cầu lớn không được xử lý sớm, chúng có thể trở nên rất khó gán ở các bước sau vì sức chứa của cơ sở đã bị chia nhỏ. Greedy 2 giải quyết vấn đề này bằng tư tưởng ưu tiên xử lý các khách hàng "khó" (có nhu cầu lớn) trước.

Khác với việc chọn cơ sở để mở trước như Greedy 1, Greedy 2 duyệt từng khách hàng theo thứ tự giảm dần của nhu cầu và chọn ra cơ sở có chi phí gán nhỏ nhất. Chi phí gán cân nhắc đồng thời cả chi phí phục vụ và chi phí cố định để mở cơ sở (nếu cơ sở đó chưa mở).

3.2.1 Ý tưởng thuật toán và các tiêu chí đánh giá "mức khó"

Tiêu chí "độ khó" trong phiên bản này được đánh giá trực tiếp qua nhu cầu d_j của khách hàng, tức là khách hàng có d_j càng lớn sẽ càng được xử lý trước. Xử lý khách hàng lớn trước giúp giảm thiểu nguy cơ tạo ra nghiệm không khả thi khi sức chứa là tài nguyên bị giới hạn.

Quy trình hoạt động của Greedy 2:

- Khởi tạo khách hàng, cơ sở và sức chứa ban đầu, sau đó sắp xếp khách hàng theo nhu cầu giảm dần.
- Xét tuần tự từng khách hàng và kiểm tra tất cả các cơ sở còn đủ sức chứa.
- Tính chi phí gán khách hàng vào từng cơ sở hợp lệ, bao gồm cả chi phí mở kho nếu kho chưa hoạt động.
- Lựa chọn cơ sở có chi phí gán nhỏ nhất, mở cơ sở (nếu cần), cập nhật chi phí tổng và sức chứa còn lại của cơ sở.
- Thuật toán dừng lại và kết luận không khả thi nếu tại một bước không còn cơ sở nào đủ sức chứa.

Chi phí gán một khách hàng j vào cơ sở i được tính bằng c_{ij} nếu cơ sở đã mở, và bằng $c_{ij} + f_i$ nếu cơ sở chưa mở. Cơ sở tốt nhất cho khách hàng j là cơ sở thỏa mãn điều kiện sức chứa và có giá trị $assignment_cost(i, j)$ đạt cực tiểu. Việc kết hợp 2 tiêu chí (ưu tiên khách hàng lớn và chọn cơ sở chi phí rẻ nhất) giúp đảm bảo tính khả thi đồng thời giữ tổng chi phí ở mức thấp.

3.2.2 Ưu điểm và nhược điểm của Greedy 2

Ưu điểm:

- Khả năng xử lý các ràng buộc về sức chứa tốt hơn hẳn bằng việc ưu tiên các khách hàng có nhu cầu lớn.
- Có thời gian chạy nhanh và đơn giản, cực kì phù hợp tạo nghiệm ban đầu cho thuật toán cải thiện.
- Việc cộng thêm chi phí cố định vào chi phí gán giúp hạn chế mở thêm cơ sở mới một cách tùy tiện.
- Cung cấp cấu trúc nghiệm rõ ràng, rành mạch để dễ dàng triển khai các phép tìm kiếm cục bộ (Move 1, 2, 3, 4 hoặc VND).
- Trong thực nghiệm, kết quả trả về của Greedy 2 thường ưu việt hơn so với Greedy 1 nhờ cách giải quyết trực tiếp ràng buộc sức chứa.

Nhược điểm:

- Thuật toán vẫn dựa trên các quyết định tham lam cục bộ tại từng bước.

- Mức độ "khó" hiện tại chỉ dựa vào nhu cầu, bỏ qua các yếu tố quan trọng khác như số lượng cơ sở có thể phục vụ hay chênh lệch chi phí.
- Hoàn toàn không có cơ chế tự điều chỉnh và chuyển đổi khách hàng sau khi đã gán.
- Đánh giá toàn bộ chi phí cố định cho khách hàng đầu tiên đôi khi gây ra sự thiên vị cho các cơ sở đã được mở sẵn.
- Chất lượng của nghiệm cuối cùng chịu sự tác động lớn từ thứ tự sắp xếp của các khách hàng có nhu cầu tương đương nhau.

Chương 4

Các phương pháp cải thiện nghiệm

4.1 Các bước di chuyển (Moves) cải thiện nghiệm

Sau khi xây dựng nghiệm ban đầu bằng thuật toán Greedy, đồ án áp dụng các bước cải thiện nghiệm theo hướng Local Search. Ý tưởng cốt lõi là xuất phát từ một nghiệm khả thi, sau đó thử thay đổi một phần nhỏ của nghiệm hiện tại. Thay đổi sẽ được chấp nhận nếu nó làm giảm tổng chi phí và vẫn thỏa mãn ràng buộc về sức chứa.

Trong bài toán Capacitated Facility Location Problem (CFLP), một cấu hình nghiệm bao gồm hai thành phần chính là tập các cơ sở đang mở và sự phân bổ khách hàng vào các cơ sở đó. Do vậy, các bước di chuyển được thiết kế xoay quanh hai thao tác: thay đổi cơ sở phục vụ khách hàng và thay đổi trạng thái đóng/mở của cơ sở.

4.1.1 Move 1: Reassignment (Gán lại khách hàng)

Move 1 thực hiện việc chuyển một khách hàng j từ cơ sở hiện tại i sang một cơ sở mới k . Điều kiện bắt buộc là cơ sở k phải còn đủ sức chứa để đáp ứng nhu cầu của khách hàng j .

Mức thay đổi chi phí Δ ban đầu được tính bằng sự chênh lệch chi phí phục vụ:

$$\Delta = c_{kj} - c_{ij}$$

Bên cạnh đó, có các trường hợp cập nhật thêm:

- Nếu cơ sở k đang đóng, thuật toán sẽ mở nó và cộng thêm chi phí cố định: $\Delta = \Delta + f_k$.
- Nếu cơ sở i chỉ phục vụ duy nhất khách hàng j , sau khi chuyển đi, cơ sở i sẽ được đóng và trừ đi chi phí cố định: $\Delta = \Delta - f_i$.

Phép di chuyển được chấp nhận nếu $\Delta < 0$, đồng nghĩa với việc tổng chi phí sau khi chuyển nhỏ hơn tổng chi phí ban đầu.

4.1.2 Move 2: Close facility (Đóng cơ sở)

Move 2 thử đóng một cơ sở i đang mở và gán lại toàn bộ khách hàng của cơ sở đó sang các cơ sở đang mở khác. Điều kiện khả thi là tất cả khách hàng phải được gán thành

công, các cơ sở nhận không bị vượt sức chứa và chỉ gán sang các cơ sở đang hoạt động.

Giá trị Δ khởi tạo bằng việc tiết kiệm chi phí mở cơ sở:

$$\Delta = -f_i$$

Sau đó, với mỗi khách hàng j được chuyển sang cơ sở k , chi phí phục vụ thay đổi được cộng dồn: $\Delta = \Delta + c_{kj} - c_{ij}$. Move này được chấp nhận nếu $\Delta < 0$, có tác dụng lớn trong việc loại bỏ các cơ sở tồn kém nhưng phục vụ ít khách hàng.

4.1.3 Move 3: Open + reassign (Mở và chuyển khách hàng)

Move 3 thử mở một cơ sở k đang đóng và chuyển các khách hàng phù hợp từ cơ sở hiện tại i sang cơ sở mới này. Ứng viên lý tưởng là các khách hàng có chi phí phục vụ tại k thấp hơn tại i , được ưu tiên gán sao cho không vượt sức chứa của k .

Giá trị Δ khởi tạo bằng chi phí mở cơ sở mới:

$$\Delta = f_k$$

Với mỗi khách hàng j chuyển sang, giá trị cập nhật là: $\Delta = \Delta + c_{kj} - c_{ij}$. Phép di chuyển được chấp nhận nếu $\Delta < 0$, giúp phát hiện các kho đóng có vị trí chiến lược cho một nhóm khách hàng.

4.1.4 Move 4: Swap (Hoán đổi trạng thái đóng/mở)

Move 4 thực hiện hoán đổi trạng thái bằng cách đóng một cơ sở i đang mở và mở một cơ sở k đang đóng. Toàn bộ khách hàng của i sẽ được chuyển sang k hoặc các cơ sở đang mở khác còn đủ sức chứa.

Công thức Δ ban đầu:

$$\Delta = f_k - f_i$$

Với mỗi khách hàng j chuyển từ i sang cơ sở mới h , độ chênh lệch tiếp tục được cộng dồn: $\Delta = \Delta + c_{hj} - c_{ij}$. Nước đi này hợp lệ khi tất cả khách hàng được gán hợp lệ và $\Delta < 0$. Nhờ khả năng thay đổi đồng thời tập cơ sở mở và phân bổ khách hàng, Move 4 mạnh mẽ hơn Move 2 và Move 3.

4.1.5 Move 5: Customer Swap (Hoán đổi hai khách hàng)

Move 5 hoán đổi vị trí cơ sở phục vụ của hai khách hàng a (tại cơ sở i) và b (tại cơ sở k) với điều kiện $i \neq k$. Các cơ sở i và k không được vượt quá sức chứa sau khi đổi.

Do không có sự thay đổi về trạng thái đóng/mở cơ sở, Δ thuần túy là độ chênh lệch chi phí phục vụ:

$$\Delta = c_{ka} + c_{ib} - c_{ia} - c_{kb}$$

Thuật toán áp dụng hoán đổi nếu $\Delta < 0$, rất hữu ích cho việc tinh chỉnh nghiệm khi tập cơ sở mở đã dần ổn định.

4.2 Sơ đồ thuật toán Local Search

Các thuật toán Local Search đều khởi chạy từ nghiệm ban đầu do Greedy 2 tạo ra. Quy trình tổng quát thực hiện lặp đi lặp lại việc tìm move cải thiện tốt nhất trong vùng lân cận; nếu tìm thấy, thuật toán áp dụng move để cập nhật nghiệm và giá trị mục tiêu, ngược lại sẽ dừng thuật toán.

Khi áp dụng Variable Neighborhood Descent (VND), thuật toán duyệt qua nhiều vùng lân cận theo thứ tự định sẵn:

- Khởi tạo biến $k = 1$ để trở đến vùng lân cận đầu tiên.
- Tìm kiếm sự cải thiện bằng Move k .
- Nếu tìm thấy cải thiện: Áp dụng move và quay lại vùng lân cận đầu tiên ($k = 1$).
- Nếu không có cải thiện: Chuyển sang vùng lân cận tiếp theo ($k = k + 1$).
- Thuật toán dừng lại khi k vượt quá số lượng move cho phép.

Chiến lược VND giúp khai thác đa dạng các kiểu biến đổi nghiệm, ngăn ngừa việc thuật toán bị mắc kẹt tại một vùng lân cận cục bộ duy nhất.

4.3 Tối ưu hóa tính toán: Phương pháp tính nhanh Delta

Trong bài toán CFLP, hàm mục tiêu bằng tổng chi phí mở cơ sở cộng với tổng chi phí phục vụ khách hàng. Nếu tính lại toàn bộ mục tiêu sau mỗi lần thử move, hệ thống sẽ phải duyệt lại toàn bộ các cơ sở và khách hàng, gây lãng phí tài nguyên khổng lồ.

Để khắc phục, phương pháp "tính nhanh Delta" được áp dụng, dựa trên nguyên tắc:

$$Objectivemi = Objectivehinti + \Delta$$

Một move chỉ được coi là cải thiện khi $\Delta < 0$.

Lợi ích của phương pháp này bao gồm: không cần tính lại toàn bộ mục tiêu, giảm số phép tính toán, giúp Local Search và VND chạy nhanh hơn, và dễ dàng so sánh các ứng viên. Tóm tắt Δ cho từng move:

- **Move 1:** Thay đổi chi phí phục vụ của 1 khách hàng và chi phí đóng/mở cơ sở (nếu có).
- **Move 2:** Chi phí cố định được hoàn lại và chênh lệch chi phí khi gán lại khách hàng cũ.
- **Move 3:** Chi phí mở cơ sở mới trừ đi tổng tiết kiệm chi phí phục vụ của tập khách hàng chuyển sang.
- **Move 4:** Chênh lệch chi phí cố định giữa kho mới và kho cũ, cộng với chi phí gán lại khách hàng.
- **Move 5:** Hoàn toàn là chênh lệch chi phí phục vụ chéo của hai khách hàng, không dính líu tới chi phí cố định.

4.4 Kết luận chương

Chương này đã đi sâu phân tích 5 bước di chuyển (Move) tác động trực tiếp lên tập cơ sở mở và phân bổ khách hàng của bài toán CFLP. Đồng thời, sơ đồ thuật toán Local Search, VND và cơ chế tính nhanh Delta đã được trình bày chi tiết nhằm giải quyết bài toán giảm thiểu thời gian đánh giá nghiệm, tạo tiền đề vững chắc cho việc thực nghiệm ở các chương tiếp theo.

Chương 5

CÀI ĐẶT THỰC NGHIỆM

5.1 Mô tả bộ dữ liệu chuẩn

Đồ án sử dụng bộ dữ liệu chuẩn **Capacitated Warehouse Location Problem** từ **OR-Library**. Đây là bộ dữ liệu thường được dùng để kiểm thử các thuật toán cho bài toán định vị cơ sở có ràng buộc sức chứa, tương đương với bài toán **Capacitated Facility Location Problem (CFLP)**.

Trong bộ dữ liệu này, mỗi instance mô tả một bài toán bao gồm các thông số:

- Số lượng cơ sở có thể mở.
- Số lượng khách hàng cần phục vụ.
- Chi phí cố định khi mở từng cơ sở.
- Sức chứa tối đa của từng cơ sở.
- Nhu cầu của từng khách hàng.
- Chi phí phục vụ từng khách hàng từ từng cơ sở.

Một nghiệm hợp lệ cần thỏa mãn các điều kiện:

- Mỗi khách hàng được gán cho đúng một cơ sở.
- Một khách hàng chỉ được gán cho cơ sở đang mở.
- Tổng nhu cầu khách hàng gán vào một cơ sở không vượt quá sức chứa của cơ sở đó.
- Tổng chi phí gồm chi phí mở cơ sở và chi phí phục vụ khách hàng là nhỏ nhất có thể.

Dữ liệu được lưu trữ trong thư mục `data_preprocessed/` với các định dạng tệp như `cap41.txt`, `cap42.txt`, ..., `cap134.txt`, `cap51.txt`. Trong chương trình, dữ liệu được đọc bằng hàm `parse_cap_file(filepath)` nằm trong tệp `readfile.py`. Sau khi xử lý, dữ liệu trả về dạng dictionary chứa các trường chính: `num_facilities`, `num_customers`, `fixed_costs`, `capacities`, `demands`, và `cost_matrix`. Cách tổ chức này giúp toàn bộ các phương pháp (MILP, Greedy, Local Search, VND) sử dụng chung một cấu trúc đầu vào.

5.2 Cài đặt mô hình Exact MILP sử dụng thư viện OR-Tools

Để có kết quả đối chứng, đồ án cài đặt mô hình chính xác bằng quy hoạch tuyến tính nguyên hỗn hợp thông qua thư viện **Google OR-Tools**.

Mô hình sử dụng hai nhóm biến quyết định chính:

- $y_i = 1$ nếu cơ sở i được mở, ngược lại bằng 0.
- $x_{ij} = 1$ nếu khách hàng j được gán cho cơ sở i , ngược lại bằng 0.

Hàm mục tiêu: Tối thiểu hóa tổng chi phí $\sum(f_i \times y_i) + \sum(c_{ij} \times x_{ij})$.

Ràng buộc gán khách hàng: $\sum_i x_{ij} = 1$ với mọi khách hàng j .

Ràng buộc sức chứa: $\sum_j (d_j \times x_{ij}) \leq s_i \times y_i$ với mọi cơ sở i . Ràng buộc này đồng thời khóa việc gán khách vào cơ sở đóng (nếu $y_i = 0$, vế phải bằng 0).

Vai trò của MILP là cung cấp nghiệm chuẩn (hoặc nghiệm đối chứng) để đánh giá chất lượng các thuật toán heuristic. OR-Tools có thể giải các instance trong thời gian ngắn, tạo cơ sở tính toán khoảng cách sai lệch. File cài đặt chính là `run_experiments.py` với hàm `solve_for_baseline(data, time_limit=60)` và kết quả được lưu tại `output/baselines_milp.csv`.

5.3 Cài đặt các thuật toán Greedy

Đồ án triển khai các thuật toán Greedy để tạo nghiệm nhanh, phục vụ làm điểm xuất phát cho Local Search và VND.

5.3.1 Greedy 1

Xây dựng nghiệm bằng cách chọn cơ sở phục vụ cho khách hàng dựa trên chi phí nhỏ nhất có xét đến khả năng chứa của cơ sở. Thuật toán duyệt qua các khách hàng, tìm cơ sở phù hợp nhất, mở cơ sở nếu cần, gán khách hàng và cập nhật sức chứa. Cài đặt tại tệp `greedy1.py`. Phương pháp này chạy nhanh nhưng quyết định mang tính cục bộ nên chất lượng nghiệm đôi khi chưa cao.

5.3.2 Greedy 2

Được chọn làm nghiệm khởi tạo chính nhờ ưu tiên xử lý các khách hàng có nhu cầu lớn trước, giảm thiểu nguy cơ khách lớn bị kẹt khi sức chứa đã bị chia nhỏ. Thuật toán sắp xếp khách hàng theo nhu cầu giảm dần, gán vào cơ sở có chi phí tăng thêm nhỏ nhất và cập nhật trạng thái. Cài đặt tại tệp `greedy2.py`.

5.4 Cài đặt thuật toán Local Search

Local Search được triển khai để cải thiện nghiệm của Greedy 2 thông qua việc đánh giá các phép biến đổi lân cận; nghiệm được cập nhật nếu tổng chi phí giảm và sức chứa được đảm bảo. Đồ án cài đặt 5 bước di chuyển:

- **Move 1 (Reassignment):** Thử chuyển một khách hàng từ cơ sở hiện tại sang cơ sở khác nhằm giảm chi phí phục vụ riêng lẻ, cài đặt trong `greedy2m1.py`.

- **Move 2 (Close facility):** Thử đóng một cơ sở đang mở và gán toàn bộ khách hàng sang các cơ sở khác nhằm loại bỏ các cơ sở có chi phí cố định lớn, cài đặt trong `greedy2m2.py`.
- **Move 3 (Open + Reassign):** Thử mở một cơ sở đang đóng và chuyển khách hàng sang nếu tiết kiệm đủ bù chi phí mở mới, cài đặt trong `greedy2m3.py` thông qua hàm `move3_local_search(data, sol)`.
- **Move 4 (Close + Open + Reassign):** Thay thế một cơ sở kém hiệu quả bằng một cơ sở tốt hơn (đóng cũ, mở mới, và gán lại), cài đặt trong `greedy2m4.py` bằng hàm `move4_swap(data, sol)`.
- **Move 5 (Customer Swap):** Hoán đổi chéo cơ sở phục vụ giữa hai khách hàng để giảm chi phí mà không thay đổi trạng thái đóng/mở của hệ thống, cài đặt trong `greedy2m5.py` thông qua hàm `move5_customer_swap(data, sol)`.

5.4.1 Thuật toán VND

Ngoài các bước đơn lẻ, Variable Neighborhood Descent (VND) kết hợp nhiều vùng lân cận theo chuỗi. Nếu Move hiện tại tạo ra cải thiện, hệ thống áp dụng và xoay vòng về Move đầu tiên; nếu không, sẽ tiến lên Move kế tiếp. Đồ án triển khai 3 phiên bản để đánh giá hiệu suất khi tăng dần vùng lân cận: VND3 (Move 1-3), VND4 (Move 1-4), và VND đầy đủ (Move 1-5), lưu tại các tệp tương ứng như `greedy2_vnd.py`.

5.4.2 Chạy thực nghiệm tổng hợp

Toàn bộ hệ thống chạy tự động thông qua tệp trung tâm `run_all_unified.py`. Quy trình sẽ khởi chạy đồng loạt các thuật toán MILP, Greedy, các nhóm Move đơn lẻ và VND, cuối cùng tập hợp mọi kết quả vào tệp duy nhất `output/results_all_unified.csv`.

5.5 Kết luận chương

Chương này đã hệ thống hóa quy trình cài đặt thực nghiệm, từ việc xử lý bộ dữ liệu OR-Library, xây dựng Baseline qua mô hình MILP, cho đến thiết lập các chiến lược Greedy và hệ thống Local Search/VND đa dạng. Việc cấu trúc tách bạch giúp đánh giá tường minh sự tiến hóa về chất lượng nghiệm từ lúc khởi tạo cho đến khi được tinh chỉnh qua nhiều vùng lân cận.

Chương 6

SO SÁNH VÀ ĐÁNH GIÁ KẾT QUẢ THỰC NGHIỆM

6.1 So sánh chất lượng nghiệm giữa Exact MILP, Greedy và Local Search

Chương này tổng hợp kết quả thực nghiệm của các thuật toán đã cài đặt cho bài toán **Capacitated Facility Location Problem (CFLP)**. Kết quả được lấy từ file output/results_all_unified với tổng số 37 instance được dùng để đánh giá.

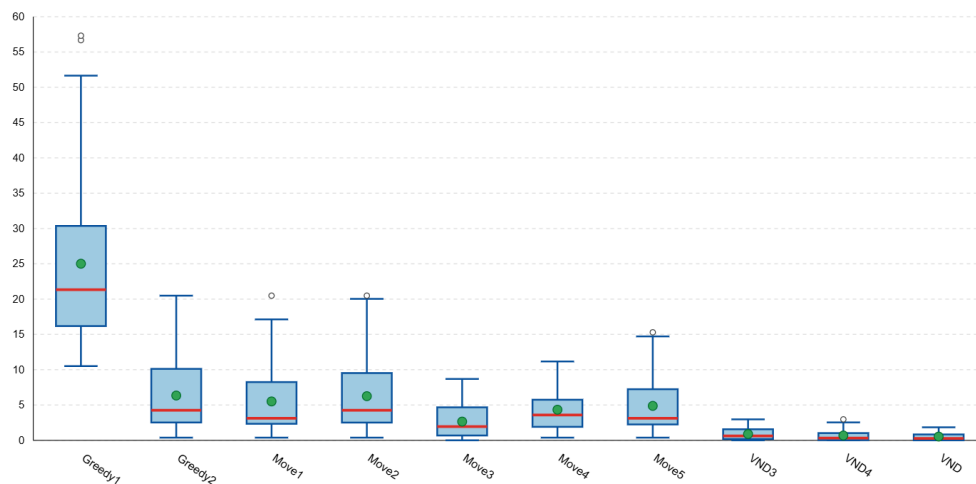
Các phương pháp được so sánh bao gồm:

- **MILP**: mô hình Exact MILP sử dụng OR-Tools.
- **Greedy1**: thuật toán tham lam thứ nhất.
- **Greedy2**: thuật toán tham lam thứ hai, được dùng làm nghiệm khởi tạo cho Local Search và VND.
- **Move1 (Reassignment)**: gán lại một khách hàng sang cơ sở khác.
- **Move2 (Close facility)**: đóng một cơ sở đang mở và gán lại khách hàng.
- **Move3 (Open + Reassign)**: mở một cơ sở đang đóng và chuyển một số khách hàng sang.
- **Move4 (Close + Open + Reassign)**: đóng một cơ sở đang mở, mở một cơ sở đang đóng và gán lại khách hàng.
- **Move5 (Customer Swap)**: hoán đổi cơ sở phục vụ giữa hai khách hàng.
- **VND3**: kết hợp Move 1, Move 2 và Move 3.
- **VND4**: kết hợp Move 1, Move 2, Move 3 và Move 4.
- **VND**: kết hợp đầy đủ Move 1 đến Move 5.

MILP cho kết quả tốt nhất vì đây là phương pháp giải chính xác, với objective trung bình là 984,851.64, được dùng làm mốc để tính gap cho các thuật toán heuristic. Trong hai thuật toán Greedy, Greedy2 tốt hơn Greedy1 rõ rệt; Greedy1 có gap trung bình 25.01%, trong khi Greedy2 chỉ còn 6.35%. Điều này cho thấy chiến lược xử lý khách hàng theo nhu

Thuật toán	Objective TB	Số cơ sở mở TB	Gap TB so với MILP
MILP	984,851.64	10.19	0.00%
Greedy1	1,222,660.52	5.32	25.01%
Greedy2	1,050,819.52	8.35	6.35%
Move1	1,041,582.58	8.24	5.50%
Move2	1,049,876.81	8.08	6.25%
Move3	1,013,344.43	11.19	2.65%
Move4	1,029,652.16	8.35	4.33%
Move5	1,034,945.37	8.35	4.88%
VND3	994,075.94	10.03	0.88%
VND4	992,066.01	10.03	0.68%
VND	990,316.40	10.00	0.53%

Bảng 6.1: So sánh chất lượng nghiệm trung bình của các phương pháp



Hình 6.1: Biểu đồ hộp (Box Plot) thể hiện phân phối Gap % của các thuật toán Heuristic so với MILP

cầu giảm dần trong Greedy2 tạo ra nghiệm ban đầu tốt hơn.

Trong các move đơn lẻ, Move3 đạt chất lượng tốt nhất với objective trung bình 1,013,344.43 và gap trung bình 2.65%. Move3 hiệu quả vì nó mở thêm một cơ sở đang đóng, sau đó chuyển các khách hàng có chi phí phục vụ thấp hơn sang cơ sở mới. Move4 đạt gap trung bình 4.33%, còn Move5 đạt 4.88%. Move4 thay đổi tập cơ sở mở bằng cách đóng một cơ sở và mở một cơ sở khác, trong khi Move5 chỉ cải thiện phân bổ khách hàng bằng thao tác hoán đổi hai khách hàng giữa hai cơ sở.

Các phiên bản VND cho kết quả tốt nhất trong nhóm heuristic. Khi tăng số lượng vùng lân cận, chất lượng nghiệm được cải thiện dần: VND3 có gap 0.88%, VND4 là 0.68% và VND đầy đủ đạt 0.53%.

Bảng 6.2 đếm số lần mỗi heuristic đạt nghiệm tốt nhất trong nhóm heuristic trên từng instance. VND đầy đủ là heuristic tốt nhất trên toàn bộ 37 instance (tổng số lần có thể lớn hơn 37 do có những instance đạt cùng giá trị objective).

Thuật toán	Số lần là heuristic tốt nhất
Greedy1	0
Greedy2	0
Move1	0
Move2	0
Move3	5
Move4	0
Move5	0
VND3	15
VND4	28
VND	37

Bảng 6.2: Thống kê số lần đạt nghiệm tốt nhất trong nhóm Heuristic

6.2 So sánh thời gian thực thi của các phương pháp

Bảng 6.3 trình bày thời gian chạy trung bình của từng phương pháp.

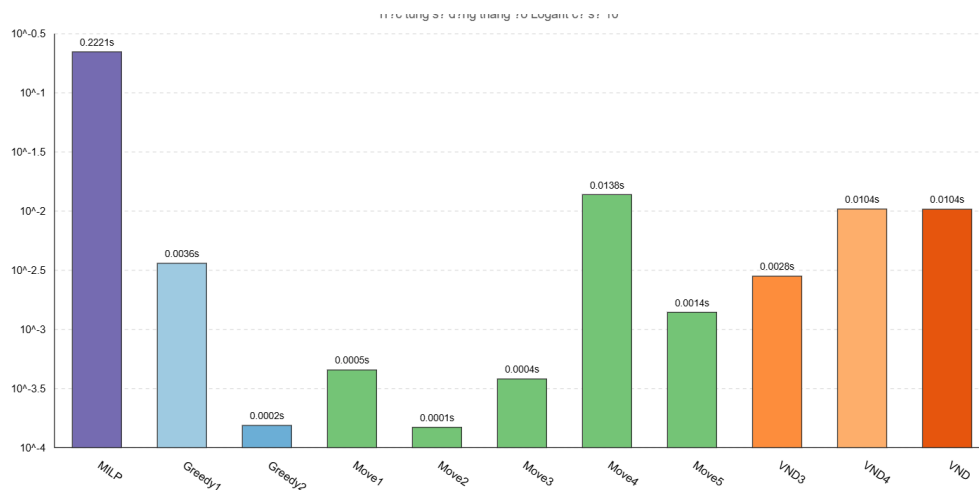
Greedy2 là phương pháp nhanh nhất với thời gian trung bình 0.0002 giây, do chỉ cần duyệt qua danh sách khách hàng và chọn cơ sở phù hợp theo tiêu chí tham lam. Các move đơn lẻ có thời gian chạy rất thấp; Move1, Move2 và Move3 đều dưới 0.001 giây, trong khi Move5 là 0.0014 giây do phải xét các cặp khách hàng để hoán đổi. Move4 có thời gian cao hơn (0.0138 giây) do phải xét đồng thời việc đóng một cơ sở, mở cơ sở khác và thử gán lại khách hàng.

MILP có thời gian trung bình 0.2221 giây; thời gian này vẫn chấp nhận được nhưng khi kích thước bài toán tăng, số lượng biến x_{ij} và ràng buộc tăng theo tích $I \times J$ sẽ khiến thời gian giải bùng nổ nhanh chóng.

Các phiên bản VND có thời gian chạy lớn hơn các move đơn lẻ vì phải kiểm tra nhiều vùng lân cận, tuy nhiên vẫn rất nhỏ: VND3 mất 0.0028 giây, VND4 và VND đầy đủ mất 0.0104 giây. Như vậy, VND đạt chất lượng nghiệm gần MILP nhưng thời gian chạy thấp hơn đáng kể.

Thuật toán	Time TB (s)
MILP	0.2221
Greedy1	0.0036
Greedy2	0.0002
Move1	0.0005
Move2	0.0001
Move3	0.0004
Move4	0.0138
Move5	0.0014
VND3	0.0028
VND4	0.0104
VND	0.0104

Bảng 6.3: So sánh thời gian thực thi trung bình



Hình 6.2: Biểu đồ so sánh thời gian thực thi trung bình (sử dụng thang đo Logarit cơ số 10)

6.3 Thảo luận về hiệu quả của các bước Local Search và phương pháp tính nhanh

6.3.1 Hiệu quả cải thiện so với Greedy 2

Do Greedy2 được dùng làm nghiệm khởi tạo, việc so sánh với Greedy2 sẽ cho thấy độ hiệu quả của từng phương pháp cải thiện (Bảng 6.4).

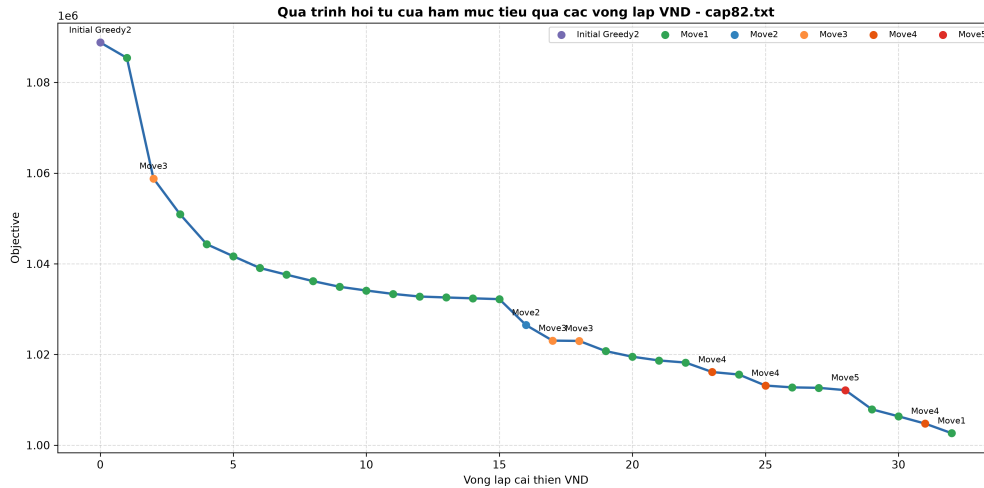
Thuật toán	Số instance cải thiện	Mức giảm objective TB	Tỷ lệ cải thiện TB
Greedy1	4	-171,841.00	-18.08%
Move1	19	9,236.94	0.77%
Move2	7	942.71	0.09%
Move3	37	37,475.08	3.37%
Move4	33	21,167.35	1.81%
Move5	24	15,874.14	1.33%
VND3	37	56,743.58	4.98%
VND4	37	58,753.51	5.17%
VND	37	60,503.12	5.30%

Bảng 6.4: Hiệu quả cải thiện nghiệm so với Greedy 2

- **Move1** cải thiện 19/37 instance (tỷ lệ 0.77%); do chỉ thay đổi vị trí một khách hàng nên mức cải thiện không lớn.
- **Move2** cải thiện 7/37 instance; vì việc đóng cơ sở yêu cầu gán lại toàn bộ khách hàng, nếu các kho còn lại thiếu sức chứa hoặc chi phí gán quá cao thì move sẽ bị loại.
- **Move3** là move đơn lẻ hiệu quả nhất, cải thiện toàn bộ 37 instance (tỷ lệ 3.37%); việc mở kho mới giúp thuật toán có thêm lựa chọn phục vụ khách hàng với chi phí tốt hơn.
- **Move4** cải thiện 33/37 instance (tỷ lệ 1.81%); mạnh hơn Move2 vì có mở kho bù đắp, nhưng chi phí tính toán cao hơn do phải gán lại toàn bộ khách hàng của kho bị đóng.
- **Move5** cải thiện 24/37 instance (tỷ lệ 1.33%); phù hợp khi tập cơ sở mở đã ổn định và chỉ cần tối ưu lại phân bổ khách hàng.

6.3.2 Hiệu quả của VND

VND kết hợp nhiều move theo thứ tự; nếu một move tìm được cải thiện, nghiệm được cập nhật và thuật toán xoay vòng về move đầu tiên, giúp khai thác đa dạng vùng lân cận. Kết quả cho thấy cả 3 phiên bản VND đều cải thiện toàn bộ 37 instance. Đặc biệt, VND đầy đủ giảm objective trung bình 60,503.12 so với Greedy2 (tương ứng tỷ lệ 5.30%), đạt kết quả tốt nhất trong các phương pháp heuristic được cài đặt.



Hình 6.3: Quá trình hội tụ (Convergence) của hàm mục tiêu qua các vòng lặp VND trên một instance tiêu biểu

6.3.3 Hiệu quả của phương pháp tính nhanh Delta

Nếu mỗi lần thử move đều tính lại toàn bộ hàm mục tiêu từ đầu, thời gian chạy sẽ tăng đáng kể. Thay vào đó, thuật toán chỉ tính phần chi phí thay đổi Δ do move gây ra: $Objective_{mi} = Objective_{hinti} + \Delta$. Nếu $\Delta < 0$, move được chấp nhận.

Cách tính Δ đặc thù cho từng move:

- **Move1:** chênh lệch chi phí phục vụ 1 khách hàng, cộng/trừ chi phí đóng/mở kho (nếu có).
- **Move2:** chi phí cố định tiết kiệm được và chi phí gán lại các khách hàng liên quan.
- **Move3:** chi phí mở kho mới và phần tiết kiệm khi chuyển khách hàng sang.
- **Move4:** chênh lệch chi phí cố định giữa hai cơ sở và chi phí gán lại khách hàng.
- **Move5:** thuần túy chênh lệch chi phí phục vụ chéo của hai khách hàng.

Nhờ tính nhanh Δ , các thuật toán có thể đánh giá hàng ngàn ứng viên cực nhanh, minh chứng bằng việc VND đầy đủ với 5 vòng lặp cận chỉ tiêu tổn trung bình 0.0104 giây.

6.4 Kết luận chương

Kết quả thực nghiệm khẳng định Greedy2 tạo nghiệm khởi tạo tốt hơn Greedy1, tạo tiền đề để Local Search hoạt động hiệu quả. Move3 đạt hiệu quả tốt nhất trong các phép biến đổi đơn lẻ với gap 2.65%. Khi tích hợp toàn bộ các toán tử vào cấu trúc VND, chất lượng nghiệm đột phá với gap chỉ 0.53% so với phương pháp chính xác MILP, trong khi thời gian thực thi được duy trì ở mức tối thiểu, biến VND thành phương pháp heuristic toàn diện nhất của đồ án.

Chương 7

KẾT LUẬN

7.1 Tóm tắt các kết quả đạt được của dự án

Đồ án đã nghiên cứu và cài đặt các phương pháp giải cho bài toán **Capacitated Facility Location Problem (CFLP)**, hay bài toán định vị cơ sở có ràng buộc sức chứa. Đây là một bài toán tối ưu tổ hợp quan trọng, trong đó cần quyết định cơ sở nào được mở và mỗi khách hàng được phục vụ bởi cơ sở nào sao cho tổng chi phí là nhỏ nhất.

Trong quá trình thực hiện, đồ án đã đạt được các kết quả chính sau:

7.1.1 Xây dựng được mô hình bài toán CFLP

Đồ án đã mô tả được các thành phần chính của bài toán, bao gồm:

- Tập các cơ sở có thể mở.
- Tập khách hàng cần được phục vụ.
- Chi phí cố định khi mở cơ sở.
- Sức chứa tối đa của từng cơ sở.
- Nhu cầu của từng khách hàng.
- Chi phí phục vụ khách hàng từ từng cơ sở.

Bài toán được mô hình hóa với mục tiêu tối thiểu hóa tổng chi phí (Bảng Chi phí mở cơ sở + Chi phí phục vụ khách hàng). Đồng thời, nghiệm phải thỏa mãn các ràng buộc:

- Mỗi khách hàng được gán cho đúng một cơ sở.
- Khách hàng chỉ được gán cho cơ sở đang mở.
- Tổng nhu cầu khách hàng tại mỗi cơ sở không vượt quá sức chứa của cơ sở đó.

7.1.2 Cài đặt mô hình Exact MILP bằng OR-Tools

Đồ án đã cài đặt mô hình giải chính xác bằng **Mixed Integer Linear Programming (MILP)** sử dụng thư viện **Google OR-Tools**. Mô hình MILP sử dụng hai nhóm biến quyết định: $y_i = 1$ nếu cơ sở i được mở và $x_{ij} = 1$ nếu khách hàng j được gán cho cơ sở i .

Mô hình Exact MILP được dùng làm cơ sở đối chứng để đánh giá chất lượng nghiệm của các thuật toán heuristic. Kết quả thực nghiệm cho thấy MILP đạt objective trung bình 984,851.64 và được xem là mốc tối ưu để tính gap cho các phương pháp khác.

7.1.3 Cài đặt các thuật toán Greedy

Đồ án đã cài đặt hai thuật toán tham lam:

- **Greedy1:** xây dựng nghiệm theo tiêu chí chọn cơ sở có chi phí phù hợp cho từng khách hàng.
- **Greedy2:** xử lý khách hàng theo nhu cầu giảm dần, sau đó chọn cơ sở có chi phí tăng thêm nhỏ nhất.

Kết quả cho thấy Greedy2 tốt hơn Greedy1 rõ rệt (Bảng 7.1). Do đó, Greedy2 được chọn làm nghiệm khởi tạo cho các thuật toán Local Search và VND.

Thuật toán	Objective TB	Gap TB so với MILP
Greedy1	1,222,660.52	25.01%
Greedy2	1,050,819.52	6.35%

Bảng 7.1: So sánh hai chiến lược Greedy

7.1.4 Cài đặt các bước Local Search

Đồ án đã xây dựng và cài đặt 5 bước di chuyển cải thiện nghiệm nhằm thay đổi phân bố khách hàng hoặc thay đổi tập cơ sở đang mở (Bảng 7.2).

Move	Tên	Ý tưởng chính
Move 1	Reassignment	Gán lại một khách hàng sang cơ sở khác
Move 2	Close facility	Đóng một cơ sở và gán lại khách hàng
Move 3	Open + Reassign	Mở một cơ sở đang đóng và chuyển khách hàng sang
Move 4	Close + Open + Reassign	Đóng một cơ sở, mở cơ sở khác và gán lại khách hàng
Move 5	Customer Swap	Hoán đổi cơ sở phục vụ giữa hai khách hàng

Bảng 7.2: Các bước di chuyển Local Search

Trong các move đơn lẻ, **Move 3** đạt hiệu quả tốt nhất với objective trung bình 1,013,344.43 (gap trung bình 2.65%). Điều này cho thấy việc mở thêm một cơ sở đang đóng và chuyển các khách hàng phù hợp sang cơ sở đó có thể giảm đáng kể chi phí phục vụ.

7.1.5 Cài đặt các phiên bản VND

Đồ án đã cài đặt thuật toán **Variable Neighborhood Descent (VND)** bằng cách kết hợp nhiều vùng lân cận (VND3, VND4, và VND đầy đủ).

Kết quả thực nghiệm cho thấy chất lượng nghiệm được cải thiện khi tăng số lượng move trong VND (Bảng 7.3). VND đầy đủ sử dụng Move 1 đến Move 5 là heuristic tốt nhất trong đồ án, đạt gap trung bình chỉ 0.53% so với MILP.

Thuật toán	Objective TB	Gap TB so với MILP
VND3	994,075.94	0.88%
VND4	992,066.01	0.68%
VND	990,316.40	0.53%

Bảng 7.3: So sánh các phiên bản VND

7.1.6 Áp dụng phương pháp tính nhanh Delta

Đồ án đã sử dụng phương pháp tính nhanh Δ để đánh giá hiệu quả của một move mà không cần tính lại toàn bộ hàm mục tiêu. Thay vì tính lại objective từ đầu, thuật toán chỉ tính phần thay đổi chi phí: $Objectivemi = Objectivehinti + \Delta$. Nếu $\Delta < 0$ thì move được xem là cải thiện và có thể được áp dụng.

Phương pháp này giúp giảm thời gian tính toán đáng kể, đặc biệt khi thuật toán phải duyệt qua nhiều move ứng viên. Kết quả thực nghiệm cho thấy dù VND đầy đủ sử dụng 5 vùng lân cận, thời gian chạy trung bình vẫn chỉ khoảng 0.0104 giây.

7.1.7 Kết quả tổng hợp

Bảng 7.4 tóm tắt kết quả chính của các phương pháp. Nhìn chung, dự án đã hoàn thành được mục tiêu xây dựng, cài đặt và so sánh nhiều phương pháp giải cho CFLP. Kết quả cho thấy VND là phương pháp heuristic hiệu quả nhất trong phạm vi đồ án.

Thuật toán	Objective TB	Time TB (s)	Gap TB so với MILP
MILP	984,851.64	0.2221	0.00%
Greedy1	1,222,660.52	0.0036	25.01%
Greedy2	1,050,819.52	0.0002	6.35%
Move3	1,013,344.43	0.0004	2.65%
VND3	994,075.94	0.0028	0.88%
VND4	992,066.01	0.0104	0.68%
VND	990,316.40	0.0104	0.53%

Bảng 7.4: Tổng hợp kết quả các thuật toán

7.2 Định hướng phát triển và tối ưu hóa thêm

Mặc dù các thuật toán đã cài đặt cho kết quả tốt trên bộ dữ liệu thực nghiệm, vẫn còn nhiều hướng có thể phát triển thêm để cải thiện chất lượng nghiệm, tốc độ xử lý và khả năng áp dụng cho các bài toán lớn hơn.

7.2.1 Cải thiện chiến lược khởi tạo nghiệm

Hiện tại, thuật toán Local Search và VND sử dụng nghiệm khởi tạo từ Greedy2. Trong tương lai, có thể thử thêm các chiến lược khởi tạo khác như: Greedy có xét tỷ lệ chi phí/sức chứa; Greedy chọn cơ sở dựa trên chi phí cố định và chi phí phục vụ trung bình; khởi tạo ngẫu nhiên có kiểm soát; hoặc tạo nhiều nghiệm ban đầu và chọn nghiệm tốt nhất.

7.2.2 Bổ sung các metaheuristic nâng cao

Một hướng phát triển quan trọng là cài đặt thêm các thuật toán metaheuristic mạnh hơn:

- **Simulated Annealing (SA):** Cho phép chấp nhận tạm thời một số nghiệm xấu hơn để thoát khỏi cực trị cục bộ.
- **Genetic Algorithm (GA):** Mô phỏng tiến hóa tự nhiên; mỗi cá thể biểu diễn tập cơ sở mở và cách gán khách hàng.
- **Tabu Search:** Sử dụng danh sách cấm để tránh quay lại các nghiệm đã xét gần đây nhằm mở rộng không gian tìm kiếm.
- **GRASP:** Kết hợp xây dựng nghiệm ngẫu nhiên có định hướng với Local Search.
- **GVNS:** Mở rộng VND bằng cách thêm bước xáo trộn nghiệm để thoát khỏi nghiệm cục bộ.

7.2.3 Tối ưu hóa tốc độ tính toán

Một số hướng tối ưu tốc độ có thể thực hiện thêm:

- Lưu trước danh sách cơ sở gần nhất của từng khách hàng.
- Giới hạn số ứng viên khi xét move hoặc chỉ xét các ứng viên có khả năng tạo cải thiện cao.
- Cập nhật incremental các thông tin như sức chứa còn lại, số khách hàng tại mỗi cơ sở.
- Tránh duyệt toàn bộ cặp khách hàng trong Move5 bằng cách lọc ứng viên trước.

7.2.4 Mở rộng bộ dữ liệu và so sánh chuyên sâu

Có thể mở rộng thực nghiệm trên các instance có kích thước lớn hơn, dữ liệu sinh ngẫu nhiên, hoặc dữ liệu thực tế về kho hàng. Ngoài OR-Tools, có thể so sánh thêm với các solver mạnh khác như SCIP, Gurobi, CPLEX, hoặc các heuristic/metaheuristic trong các nghiên cứu liên quan. Bên cạnh đó, bổ sung các biểu đồ trực quan (gap trung bình, thời gian chạy, mức cải thiện) sẽ giúp phân tích trở nên thuyết phục và dễ tiếp cận hơn.

7.3 Kết luận chung

Đồ án đã hoàn thành việc nghiên cứu và cài đặt nhiều phương pháp giải cho bài toán CFLP, từ mô hình chính xác MILP đến các thuật toán heuristic như Greedy, Local Search

và VND. Kết quả thực nghiệm cho thấy:

- Greedy2 tạo nghiệm ban đầu tốt hơn Greedy1.
- Các bước Local Search giúp cải thiện nghiệm Greedy2, trong đó Move3 là move đơn lẻ hiệu quả nhất.
- VND kết hợp nhiều vùng lân cận cho chất lượng nghiệm tốt nhất; VND đầy đủ đạt gap trung bình chỉ 0.53% so với MILP.
- Phương pháp tính nhanh Delta giúp thuật toán chạy nhanh dù phải xét nhiều move ứng viên.

Từ các kết quả trên, có thể kết luận rằng việc kết hợp nghiệm khởi tạo Greedy2 với các bước cải thiện Local Search và VND là hướng tiếp cận hiệu quả cho bài toán định vị cơ sở có ràng buộc sức chứa.